Build Your First AI Agent Workflow

A gentle, hands-on tutorial that walks a complete first-timer through a small lead-enrichment project — the smallest real thing that teaches the whole pattern.

by d.avebrown

Most people try to learn AI agents by starting with something wildly too ambitious. They want the agent to read the whole inbox, update the CRM, write every follow-up email, score the deals, book the meetings, and — while it's in there — spiritually heal the sales team. It's an understandable instinct. The demos you see online are always the flashy, end-to-end, "look, no hands" version, and it feels like anything smaller isn't really *agents*.

That is exactly the wrong place to start, and this guide is going to talk you out of it. Your first agent workflow should be boring. Genuinely, almost disappointingly boring — because boring is what you can actually see. Boring is what you can debug. And when your first workflow fails, which it absolutely will, boring is what lets you walk straight to where the body is buried instead of staring at a black box wondering what went wrong.

So we're going to build one small thing together: a lead-enrichment workflow. You'll start with a spreadsheet of business names. The workflow will research each business, summarize what it does, try to surface a likely owner or key contact when that's publicly available, and write the results back into the spreadsheet. That's the whole project. No giant CRM integration, no fully autonomous outbound machine, no "set it and forget it." Just a small, legible workflow that happens to teach you the core pattern behind every serious piece of agent-led work you'll ever build.

START HERE

Lead enrichment is just the excuse.

Here's the reframe that makes the whole tutorial click: this project is not really about leads. Lead enrichment is simply a convenient, honest little task that forces you to touch every part of an agent workflow at least once. What you're actually learning is a repeatable shape — one that shows up in support automation, document processing, research assistants, data cleanup, and a hundred other places you'll encounter later.

That shape is worth stating plainly, because you'll come back to it every time you build something new. You take structured input. You give the AI access to a tool or two. You let it perform the task one step at a time. You force its output into a predictable format. You review the results before you trust them. And then you improve the workflow based on exactly where it broke. Master that loop on a boring task, and you can point it at almost anything.

> **WHY THIS IS THE RIGHT FIRST BUILD**
>
> *Anthropic's guidance on building agents is blunt about this: start with the simplest architecture that works, and add complexity only when the task genuinely demands it. They draw a clean line between predictable workflows — where your code controls the path — and more autonomous agents, where the model decides what to do next. For your first project you want the former: a simple workflow with a couple of agentic steps living inside it.*

THE SHAPE OF IT

Spreadsheet in, richer spreadsheet out.

Before we write a single instruction, let's look at the two ends of the workflow, because everything else is just the bridge between them. You start with a small file — Excel or CSV, it doesn't matter — holding a list of leads. A few columns, most of them possibly blank:

Company Name	Website	City	State
Acme Glass Co.		Albany	NY
Northside Electric		Rochester	NY
Summit Roofing LLC		Buffalo	NY

And you end with the same file, enriched — the original rows carried forward, plus a handful of new columns the workflow filled in for you:

Company Summary	Website Found	Likely Owner / Contact	Source Notes	Confidence	Needs Review
Residential & commercial glass repair, Albany area.	example.com	—	Site found; owner unclear.	medium	yes

Notice what the goal is *not*. The goal is not flawless, hands-off automation. The goal is a first working version that helps you research leads meaningfully faster than doing every row by hand — and, more importantly, one you understand end to end. Perfect can come later. Legible comes first.

THE MENTAL MODEL

Most of your "AI agent" isn't AI at all.

When people hear "AI agent," they tend to picture a little digital employee running around the internet making brilliant, independent decisions. For your first project, throw that image out. A far more useful — and more accurate — picture is an assembly line where the AI handles only the one or two judgment-heavy steps, and ordinary, boring code handles everything else.

Walk down the line and you'll see how little of it is actually the model. The piece that loads the spreadsheet is not AI. The piece that runs a web search is not AI. The piece that writes results back into the file is not AI. The model sits in the middle of all that plumbing, and its job is narrow: decide what to search for, read the messy results that come back, and turn them into clean, structured fields. That's it. That's the whole contribution.

> **THE LOOP AT THE HEART OF IT**
>
> *OpenAI's tool-calling docs describe the mechanism as a loop, and it's worth memorizing: you give the model tools it can call, the model asks to call one, your code runs that tool, and you hand the result back so the model can keep going or produce its final answer. That's the engine of agentic work. Not vibes. Not magic. A loop.*

THE ARCHITECTURE

A prompt chain — the simplest thing that could possibly work

One row, one predictable sequence, then repeat.

For this first build we're going to use the plainest architecture there is: a prompt chain. You take one row and walk it through a fixed sequence of steps, in order, every single time. There's no clever branching, no swarm of agents debating each other — just a straight line you could draw on a napkin.

Read a row from the spreadsheet. Ask the model what search query it wants to run for that row. Run the search. Hand the results back to the model. Ask it to produce a structured enrichment record. Validate that record. Write it back to the spreadsheet. Move to the next row. Eight steps, always in the same order.

You could, eventually, make this far more sophisticated — routing easy leads down a cheap path and hard ones down a deep one, running research agents in parallel, adding an evaluator that grades each result, inserting human-approval checkpoints. Anthropic's own agent materials catalog exactly these patterns: sequential chains, parallelization, routing, evaluator-optimizer loops. But every one of those is an upgrade you earn *after* the simple version works. For a first lead-enrichment build, sequential is unambiguously the right move. Resist the urge to skip ahead.

THE AGENT'S JOB

Give it a narrow mandate — and permission to be uncertain

A confident intern with internet access is a liability.

The agent's job description should be almost comically narrow: research one business lead and return a short, structured enrichment record using only publicly available information — and if the information is uncertain, say so out loud. That last clause is the one that separates a useful workflow from a hallucination machine with a spreadsheet-export button, so let's sit with it for a second.

Beginner workflows go sideways in a very specific way. Someone asks the model to "find the owner," the model dutifully produces a name, and that name gets treated as fact — even though the model was really just making a plausible guess to satisfy the request. The fix isn't to forbid guessing entirely; for a demo it's perfectly fine to ask the model to look for *likely* ownership or leadership. The fix is to make the output honest about its own certainty. Every record should cleanly separate what's confirmed from public sources, what's likely-but-uncertain, what's simply missing, and what needs a human to take a look before anyone acts on it.

STRUCTURED OUTPUT

Don't let the model freestyle the result.

This is one of the first real lessons of agent work, and it's easy to skip right up until it bites you: if a downstream program needs to use the model's output, that output needs structure. A lovely paragraph of prose is useless to a spreadsheet. What a spreadsheet wants is fields — the same fields, in the same shape, every single time — and the way you guarantee that is by handing the model a schema and requiring it to fill that schema in.

OpenAI's Structured Outputs feature exists for exactly this. It forces the model's response to conform to a JSON schema, so required keys can't quietly go missing and invalid values can't sneak in. For our workflow, a reasonable enrichment schema looks like this:

ENRICHMENT SCHEMA · JSON

```
{
  "company_name":            "string",
  "website":                 "string or null",
  "business_summary":        "string",
  "likely_owner_or_contact": "string or null",
  "contact_role":            "string or null",
  "source_notes":            "string",
  "confidence":              "high | medium | low",
  "needs_human_review":      "boolean",
  "reason_for_review":       "string or null"
}
```

This isn't a fussy technical detail — it's the thing that makes the whole workflow manageable. Without a schema you get a cute paragraph you then have to parse by hand. With a schema, you get data you can drop straight back into Excel. That's the entire difference between a toy and a tool.

SCOPE DISCIPLINE

Run it on five rows, not five thousand

The first version should only process five rows.

Please do not point this at two thousand leads on day one. Run it on five. It sounds almost too cautious, but five rows is enough to surface nearly every problem you're going to hit at scale: company names too generic to pin down, businesses with several locations, search results for entirely the wrong company, missing websites, stale directory listings, ownership that simply isn't public, and — the classic — a model that sounds far more confident than the evidence warrants.

This is also where the "agent manager" mindset quietly begins. You are not merely a person using AI here. You are supervising a workflow: watching where it stumbles, tightening the instructions, sharpening the tools, and making deliberate calls about what should be automated versus what a human should still eyeball. Five rows is your control room. Two thousand rows is just noise you can't learn from yet.

THE BUILD, STEP BY STEP

Six moves that take you from empty file to working workflow

1. Create a painfully small input file

Start with a tiny spreadsheet — a few columns, a handful of rows. The website column can be blank; if you already know a company's site, include it, and if you don't, the workflow will try to find it. The point of this file is not scale, it's legibility: something small enough that you can hold every row in your head while you watch the workflow run against it.

LEADS.CSV

```
company_name,city,state,website
Acme Glass Co.,Albany,NY,
Northside Electric,Rochester,NY,
Summit Roofing LLC,Buffalo,NY,
```

2. Give the agent a narrow instruction

Now write the system instruction — the standing job description the model carries into every row. Notice, as you read it, how much of this is about restraint. It doesn't say "find me the owner no matter what." It says: find what you honestly can, flag what you can't, and never fake certainty.

SYSTEM INSTRUCTION

```
You are a lead enrichment research assistant.
Enrich one business lead at a time using only
publicly available information. For each lead:

1. Identify the most likely official website.
2. Summarize what the business does in one or
   two plain-English sentences.
3. Identify a likely owner, founder, president,
   or key contact ONLY if public info supports it.
4. If ownership is unclear, return null and mark
   the row for human review.
5. Do not guess.
6. Prefer official sites, business profiles, state
   listings, LinkedIn company pages, reputable
   directories, and local news.
7. Return the result using the required JSON schema.
```

That single instruction is the difference between a useful research assistant and a very confident intern who will happily invent a plausible owner to make you happy.

3. Give it exactly one tool: search

Resist the temptation to hand the agent a toolbox. For this first version it needs precisely one tool — a search function — and nothing more. OpenAI's docs describe tools broadly as ways to extend a model with web search, file search, function calling, remote MCP servers, and so on; you'll get to those eventually, but not today. Today you have `search_web(query)`, which takes a query string and returns a short list of results.

In practice, the model looks at a row and generates a query like `"Acme Glass Co. Albany NY owner"` or `"Acme Glass Co. Albany NY official website"`. Your code runs that search and passes the results back into the loop. One tool, one job. You can always add more once the one-tool version genuinely works.

4. Ask for the structured enrichment

Once the model has search results in hand, you make the second call — the one that turns messy findings into a clean record. This is where the schema from earlier does its real work. You give the model the lead, the search results, and a firm instruction to return only valid JSON:

ENRICHMENT PROMPT

```
Using the lead and search results below, create a
lead enrichment record.

Lead:
  Company: Acme Glass Co.
  City: Albany   State: NY   Website: unknown

Search results:
  [ ...results here... ]

Return ONLY valid JSON matching the schema.
If the owner or key contact is not clearly
supported, use null. If there is any uncertainty,
set confidence to "low" or "medium" and set
needs_human_review to true.
```

What you're steering away from is the friendly paragraph — "Acme Glass seems to be a family-owned glass company in Albany…" — and toward something a program can actually consume:

WHAT YOU WANT BACK · JSON

```
{
  "company_name": "Acme Glass Co.",
  "website": "https://example.com",
  "business_summary": "Acme Glass Co. appears to
     provide residential and commercial glass repair
     and installation in the Albany area.",
  "likely_owner_or_contact": null,
  "contact_role": null,
  "source_notes": "Official website found, but
     ownership was not clearly available in results.",
  "confidence": "medium",
  "needs_human_review": true,
  "reason_for_review": "Owner or key contact could
     not be confirmed from public sources."
}
```

That object can go straight back into a spreadsheet, row for row. That's the win you're building toward.

5. Treat the confidence field as your control system

The `confidence` field is not decoration, and it's not the model being polite. It's the dial that decides what a human ever has to look at. Keep it to three honest levels: **high** when an official website or several reliable sources line up, **medium** when it's a likely match with some gaps, and **low** when the company is ambiguous, the sources are weak, or there's a real chance of a mismatch.

Then wire that field to a couple of dead-simple rules, enforced in your code rather than left to the model's goodwill. If confidence is low, mark the row for review. If the likely owner came back null, mark it for review. If the company match itself feels uncertain, mark it for review. This is your first real taste of managing an agent: the machine does the repetitive work, and the human's attention gets spent only on the risky edges where it's actually worth something.

> **THE RULE OF THUMB**
>
> *Enforce the consequential rules in code, not in the prompt. "Please mark uncertain rows for review" is a suggestion the model can forget. An* if*-statement that sets needs_human_review whenever confidence is low is a guarantee. When a behavior has to happen, make it happen in code.*

6. Write the results back to the spreadsheet

The last step closes the loop. Once the JSON validates, you map its fields back into columns and write them into your file. Nothing fancy — just the enriched rows landing where you can read them:

Company	Business Summary	Website	Likely Contact	Confidence	Review
Acme Glass Co.	Residential & commercial glass repair, Albany.	example.com	—	medium	yes
Northside Electric	Electrical contractor, residential & commercial.	example.com	Jane Smith	high	no

And that's genuinely it. Spreadsheet in, a research step, a moment of model judgment, structured output, spreadsheet out, and human review wherever the confidence dial says so. It isn't glamorous. But that exact pattern — those exact six pieces — is reusable almost everywhere you'll go next.

MAKE IT INSPECTABLE

If you can't see what the agent did, you can't fix it.

Here's a small habit that will make this feel like real agent work rather than a party trick: log every run. For each row, quietly save the company name, the search query the model chose, the sources it considered, the final JSON it produced, the confidence, the review flag, any error message, and a timestamp. It feels like busywork right up until the first time a result looks wrong and you can pull up exactly what the agent saw and decided.

This is the beginner-friendly version of something the serious tools formalize — OpenAI's Agents SDK, for instance, ships tracing that records model generations, tool calls, handoffs, and guardrail events across a whole run. You don't need any of that yet. A plain CSV log does the job. The principle is what matters, and the principle is simple: make the agent's work inspectable.

RUN LOG · ONE LINE PER ROW

```
company_name, search_query_used, sources_considered,
final_json_output, confidence, needs_human_review,
error_message, timestamp
```

THE CURRICULUM

The failures aren't bugs in your learning. They are the learning.

A handful of things will break almost immediately, and that's not a sign you did it wrong — it's the entire point of starting with five rows. Each failure has a clean fix, and working through them is how you actually develop the instincts the role rewards.

When the company name is too generic — "Summit Roofing" might exist in twenty states — the fix is to fold city and state into the search query so the model isn't guessing between strangers. When it locks onto the *wrong* company, require it to compare name, location, and website before it accepts a match. When it invents an owner out of thin air, tighten the instruction so it returns null unless public sources clearly support a name.

When the summaries drift into fluffy marketing language, constrain them to one or two sentences and explicitly ban the marketing voice. When the output comes back inconsistent from row to row, that's your cue to lean harder on structured outputs instead of free-form text. And when it all feels too slow or too expensive, process fewer rows, cache search results you've already fetched, or route the easy leads to a cheaper model and save the strong model for the ambiguous ones.

> **THE REFRAME**
>
> *This is precisely why the first version should be tiny. You are not trying to prove the agent is brilliant. You are trying to find out, cheaply and quickly, exactly where it's dumb — because that map is what you'll be managing against forever.*

THE REVIEW PASS

Read every one of the first five results with your own eyes.

After that first run of five rows, don't rush to scale — sit down and grade each result yourself. Did it identify the right business? Did it find the official website? Is the summary actually accurate? Did it resist guessing the owner when it shouldn't have? Did it flag the uncertain rows for review? Did the JSON validate cleanly? And the one that really matters: would you trust this result enough to put it in front of a salesperson?

Wherever the answer is no, improve the workflow — and improve *this* workflow, the simple one. Do not respond to a weak result by immediately adding more agents, bolting on memory, or wiring up a CRM integration. Those are tempting, and they feel like progress, but they mostly hide the problem under more moving parts. Fix the basic loop first. A clean, boring loop you understand beats an impressive one you don't.

WHERE TO GO NEXT

Earn each layer of complexity.

Once the plain prompt chain runs reliably, you have a real foundation to build on — and now the fancier patterns will actually make sense to you, because you'll know what problem each one solves. Here's the ladder, in the order it's worth climbing.

Version 1 — the prompt chain. One row, one predictable sequence. This is the version you just built, and it's the best possible teacher.

Version 2 — add an evaluator. A second model reviews each enrichment record and flags the weak ones. This is the evaluator-optimizer pattern: one model produces, another critiques.

Version 3 — add routing. Send easy leads down a cheap, fast path and ambiguous ones down a deeper research path, so you spend effort where it pays off.

Version 4 — go parallel. One worker hunts for the website, another for ownership, another writes the summary, and a final step stitches their findings together.

Version 5 — add human approval. Before any owner or contact data gets written into a real CRM, require a person to sign off on the uncertain rows.

Version 6 — connect real systems. Eventually this can plug into a CRM, an enrichment API, Google Sheets, Airtable, HubSpot, Salesforce, or your own internal databases.

> **WHERE MCP COMES IN**
>
> *That last step is where MCP-style integrations start to matter. MCP is an open standard for connecting AI applications to external systems — data sources, tools, and workflows — giving agents a consistent way to reach the systems around them. It's genuinely useful. It's also absolutely not where you start. Start with the spreadsheet.*

The Takeaway

Your first agent workflow should not be impressive. It should be understandable. A small lead-enrichment build looks modest, but it exercises every muscle that matters: how to give a model a narrow job, how to give it a tool, how to keep the whole thing constrained, how to force structured output, how to review uncertainty honestly, and how to improve the system after you've watched it fail. That's the real unlock, and none of it requires a giant, autonomous, jaw-dropping demo.

Because the job that's actually emerging isn't "person who writes one clever prompt." It's the person who can look at a messy business process and say, with confidence: this part should be deterministic code, this part needs AI judgment, this part needs a tool, this part needs human review, and the whole thing needs to be measured. That's how you stop merely playing with AI and start managing agent-led work. And it starts, quietly, with a spreadsheet of business names.

Want more build-along guides for people new to AI agents?

Follow along on Instagram for weekly, beginner-friendly breakdowns.

@davebrownbrand